

Parte A:

Bloco	Tipo	Função	Entradas	Saídas
PC (Program Counter)	S	Armazena o endereço da instrução atual. A cada ciclo recebe o próximo endereço calculado e o mantém até o próximo clock.	Próximo PC	Endereço atual da instrução
Instruction Memory	C	Recebe o endereço vindo do PC e retorna a instrução armazenada naquele endereço. Os campos da instrução são distribuídos para os demais blocos.	Endereço (PC)	Instrução [31:0]
Adder	C	Soma 4 ao valor do PC para obter o endereço sequencial da próxima instrução.	PC e constante 4	PC + 4
Control Unit	C	Decodifica o opcode da instrução e gera sinais de controle para o datapath. Determina operações de memória, ALU, saltos e escrita em registradores.	Instruction[6:0] (opcode), às vezes funct3	RegWrite, ALUSrc, MemRead, MemWrite, Branch, Jump, etc
Registers	C	Armazena os registradores do processador. Permite ler dois registradores simultaneamente e escrever em um terceiro.	rs1 = Instruction[19:15], rs2 = Instruction[24:20], rd = Instruction[11:7], dado de escrita, RegWrite	Read Data 1, Read Data 2

ImmGen	C	Extraí e estende o valor imediato da instrução conforme o formato (I, S, B, J, etc.).	Instruction[31:0]	Imediato de 32 bits
MUX (ALUSrc)	C	Escolhe a segunda entrada da ALU: registrador (Read Data 2) ou imediato gerado pelo ImmGen.	Read Data 2, imediato, ALUSrc	Operando da ALU
ALU Control	C	Define qual operação a ALU executará com base no tipo da instrução.	ALUOp, Instruction[14:12] (funct3), Instruction[31:25] (funct7)	Sinal de operação da ALU
ALU	C	Executa operações aritméticas e lógicas, como soma, subtração, AND e OR. Também gera sinal de comparação (Zero).	Dois operandos, controle da ALU	Resultado da ALU, Zero
Data Memory	S	Realiza leitura e escrita de dados na memória. Usada por instruções de load/store.	Endereço (ALU Result), dado para escrita, MemRead, MemWrite	Dado lido da memória
MUX (MemtoReg / WriteBack)	C	Escolhe qual valor será escrito no registrador: resultado da ALU ou dado vindo da memória.	Resultado da ALU, dado da memória	Dado de escrita no banco de registradores
Adder (Branch/Jump Target)	C	Calcula o endereço de destino de branch ou jump somando PC com imediato deslocado.	PC, imediato	Endereço alvo
Jump Control	C	Decide se o próximo PC será sequencial (PC+4) ou	Branch, Jump, Zero	Controle do MUX do PC

		endereço de branch/jump. Usa sinais da unidade de controle e o resultado Zero da ALU.		
MUX (Próximo PC)	C	Seleciona o próximo valor do PC entre PC+4 e endereço de branch/jump.	PC+4, endereço alvo, controle de salto	Próximo PC

Parte B:

1. Instrução Tipo R (Aritmética): add t4, a0, s6
 Esta instrução soma os valores dos registradores a0 e s6 e salva em t4.
 (Registradores: t4 = x29, a0 = x10, s6 = x22)
 Conversão para Binário:
 0000000 (funct7) | 10110 (rs2=x22) | 01010 (rs1=x10) | 000 (funct3) | 11101 (rd=x29) | 0110011 (opcode)
 Localização no Barramento (Instruction Memory):
 [6-0]: 0110011 (Vai para Control)
 [11-7]: 11101 (Vai para Write Register)
 [14-12]: 000 (Vai para ALU Control)
 [19-15]: 01010 (Vai para Read Register 1)
 [24-20]: 10110 (Vai para Read Register 2)
 Blocos em Funcionamento: PC, Instruction Memory, Control, Registers, ALU Control, ALU, multiplexadores da saída.
 Sinais de Controle (Datapath):
 RegWrite = 1 (Permite gravar o resultado em t4).
 ALUSrc = 0 (Mux seleciona o dado do registrador Read Data 2).
 WriteSrc = 0 (Mux seleciona a saída ALU Result para gravar no registrador).
 MemRead = 0, MemWrite = 0, BranchZero = 0.

2. Instrução Tipo I (Aritmética com Imediato): addi s6, s6, 1
 Esta instrução soma o valor do registrador s6 com o número imediato 1.
 (Registradores: s6 = x22)
 Conversão para Binário:
 0000000000001 (imediato=1) | 10110 (rs1=x22) | 000 (funct3) | 10110 (rd=x22) | 0010011 (opcode)
 Localização no Barramento:
 [6-0]: 0010011 (Control)

[11-7]: 10110 (Write Register)

[19-15]: 10110 (Read Register 1)

[31-0]: A instrução vai para o Imm Gen para extrair o 000000000001 e expandir para 32 bits.

Blocos em Funcionamento: PC, Instruction Memory, Control, Registers, Imm Gen, ALU Control, ALU.

Sinais de Controle (Datapath):

RegWrite = 1.

ALUSrc = 1 (Mux escolhe o valor expandido que sai do Imm Gen em vez do Read Data 2).

WriteSrc = 0 (Mux direciona o ALU Result para os registradores).

3. Instrução Tipo I (Load): lw s10, 0(t4)

Esta instrução acessa a memória no endereço apontado por t4 e carrega o dado em s10.

(Registradores: s10 = x26, t4 = x29)

Conversão para Binário:

000000000000 (imediato=0) | 11101 (rs1=x29) | 010 (funct3) | 11010 (rd=x26) | 0000011 (opcode)

Localização no Barramento:

[6-0]: 0000011 (Control)

[11-7]: 11010 (Write Register)

[19-15]: 11101 (Read Register 1)

Blocos em Funcionamento: PC, Instruction Memory, Control, Registers, Imm Gen, ALU, Data Memory.

Sinais de Controle (Datapath):

MemRead = 1 (Habilita a leitura da memória de dados).

ALUSrc = 1 (A ALU soma t4 + 0 para calcular o endereço).

RegWrite = 1 (Habilita a gravação no banco de registradores).

WriteSrc = 1 (Mux final seleciona Read Data da memória, e não a saída da ALU).

4. Instrução Tipo S (Store): sw t0, 0(s0)

Salva o conteúdo de t0 na memória, no endereço apontado por s0.

(Registradores: t0 = x5, s0 = x8)

Conversão para Binário:

0000000 (imm[11:5]) | 00101 (rs2=x5) | 01000 (rs1=x8) | 010 (funct3) | 00000 (imm[4:0]) | 0100011 (opcode)

Localização no Barramento:

[6-0]: 0100011 (Control)

[19-15]: 01000 (Read Register 1)

[24-20]: 00101 (Read Register 2)

Blocos em Funcionamento: PC, Instruction Memory, Control, Registers, Imm Gen, ALU, Data Memory.

Sinais de Controle (Datapath):

MemWrite = 1 (Habilita a escrita na Data Memory).

ALUSrc = 1 (A ALU soma a base s0 com o offset para achar o endereço da memória).

RegWrite = 0 (Nada retorna para os registradores).

Repare que a saída Read Data 2 (t0) vai diretamente para o pino Write Data da memória.

5. Instrução Tipo B (Branch): beq t5, zero, mult_done

Compara t5 com zero. Se forem iguais, desvia o fluxo do código.

(Registradores: t5 = x30, zero = x0. Vamos assumir um offset genérico de salto de 12 bytes = 000000001100 em binário).

Conversão para Binário:

0000000 (imm) | 00000 (rs2=x0) | 11110 (rs1=x30) | 000 (funct3) | 01100 (imm) | 1100011 (opcode)

Localização no Barramento:

[6-0]: 1100011 (Control)

[19-15]: 11110 (Read Register 1)

[24-20]: 00000 (Read Register 2)

Blocos em Funcionamento: Todos os blocos da parte superior entram em ação (Add Sum, Jump Control).

Sinais de Controle (Datapath):

BranchZero = 1 (Sinaliza que é um salto condicional do tipo BEQ).

ALUSrc = 0 (A ALU recebe os dois registradores e faz uma subtração para comparar).

A flag Zero da ALU acende (vai para Jump Control). O bloco Add Sum (lá em cima) soma o PC atual com o offset do Imm Gen e o Multiplexador 0 (Mux 0 do topo) atualiza o PC.

RegWrite = 0, MemRead = 0, MemWrite = 0.

Parte C:

Algoritmo de busca linear:

Tive de modificar para conseguir executar no site (segue código alternativo de Linear Search):

main:

```
# escrever vetor na pilha já que não tem .data
```

```
addi s0, s0, 0x100 # endereço base da memória
```

```
li t0, 5 # tamanho array
```

```
sw t0, 0(s0)
```

```
li t0, 5
```

```
sw t0, 4(s0)
```

```
li t0, 4
```

```
sw t0, 8(s0)
```

```
li t0, 3
```

```
sw t0, 12(s0)
```

```
li t0, 2
```

```
sw t0, 16(s0)
```

```
li t0, 1
```

```
sw t0, 20(s0)
```

```
func:
```

```
# base array
```

```
addi a0, s0, 4
```

```
# a1 = tamanho do vetor
```

```
lw a1, 0(s0)
```

```
# a2 = valor a ser buscado
```

```
li a2, 3
```

```
# -----
```

FUNÇÃO: ls (linear search)

a0 = endereço do vetor

a1 = tamanho do vetor

a2 = valor buscado

return: a0 = índice ou -1

ls:

add t0, zero, zero

ls_loop:

bge t0, a1, ls_not_found

slli t1, t0, 2

add t1, a0, t1

lw t2, 0(t1)

beq t2, a2, ls_found

addi t0, t0, 1

j ls_loop

ls_found:

add a0, t0, zero

j fim

ls_not_found:

addi a0, zero, -1 # return -1

fim:

j fim # encerra programa

Mnemônico	Machine Code (Hex)	Machine Code (Bin)	Tip o	Active Control Bits	Mux selectors
addi s0, s0, 0x100	0x1004041 3	000100000000010000000100000100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
li t0, 5	0x0050029 3	0000000001010000000000010100100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]

sw t0, 0(s0)	0x0054202 3	000000000101010000100000001000 11	S	MemWrite e ALUSrc	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
li t0, 5	0x0050029 3	0000000001010000000000010100100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
sw t0, 4(s0)	0x0054222 3	000000000101010000100010001000 11	S	MemWrite e ALUSrc	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
li t0, 4	0x0040029 3	0000000001000000000000010100100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
sw t0, 8(s0)	0x0054242 3	000000000101010000100100001000 11	S	MemWrite e ALUSrc	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
li t0, 3	0x0030029 3	0000000000110000000000010100100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]

sw t0, 12(s0)	0x0054262 3	000000000101010000100110001000 11	S	MemWrite e ALUSrc	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
li t0, 2	0x0020029 3	000000000010000000000010100100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
sw t0, 16(s0)	0x0054282 3	000000000101010000101000001000 11	S	MemWrite e ALUSrc	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
li t0, 1	0x0010029 3	000000000001000000000010100100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
sw t0, 20(s0)	0x00542A2 3	000000000101010000101010001000 11	S	MemWrite e ALUSrc	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
addi a0, s0, 4	0x0044051 3	0000000001000100000000101000100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]

lw a1, 0(s0)	0x0004258 3	000000000000010000100101100000 11	I	MemRead, WriteSrc, ALUSrc e ReqWrite	ALUSrc [1] MemtoReg [1] PCSrc [0] Jalr [0]
li a2, 3	0x0030061 3	000000000001100000000001100001001 1	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
add t0, zero, zero	0x000002B 3	0000000000000000000000010101100 11	R	ALUOp e RegWrite	ALUSrc [0] MemtoReg [0] PCSrc [0] Jalr [0]
bge t0, a1, ls_not_found	0x02B2D26 3	0000001010110010110100100110001 1	B	BranchZero e ALUOp	ALUSrc [0] MemtoReg [0] PCSrc [0] Jalr [0]
slli t1, t0, 2	0x0022931 3	000000000010001010010011000100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
add t1, a0, t1	0x0065033 3	0000000001100101000000110011001 1	R	ALUOp e RegWrite	ALUSrc [0] MemtoReg [0] PCSrc [0] Jalr [0]

lw t2, 0(t1)	0x0003238 3	0000000000000011001000111000001 1	I	MemRead, WriteSrc, ALUSrc e ReqWrite	ALUSrc [1] MemtoReg [1] PCSrc [0] Jalr [0]
beq t2, a2, ls_found	0x00C3866 3	0000000011000011100001100110001 1	B	BranchZero e ALUOp	ALUSrc [0] MemtoReg [0] PCSrc [0] Jalr [0]
addi t0, t0, 1	0x0012829 3	000000000001001010000010100100 11	I	ALUOp, ALUSrc e RegWrite	ALUSrc [1] MemtoReg [0] PCSrc [0] Jalr [0]
j ls_loop	0xFE9FF06 F	111111010011111111000001101111	J	Jump, WriteSrc e RegWrite	ALUSrc [0] MemtoReg [2] PCSrc [1] Jalr [0]
add a0, t0, zero	0x0002853 3	000000000000001010000101001100 11	R	ALUOp e RegWrite	ALUSrc [0] MemtoReg [0] PCSrc [0] Jalr [0]
j fim	0x0080006 F	000000001000000000000000110111 1	J	Jump, WriteSrc e RegWrite	ALUSrc [0] MemtoReg [2] PCSrc [1] Jalr [0]
j fim	0x0000006 F	000000000000000000000000110111 1	J	Jump, WriteSrc e RegWrite	ALUSrc [0] MemtoReg [2]

					PCSrc [1] Jalr [0]
--	--	--	--	--	-----------------------